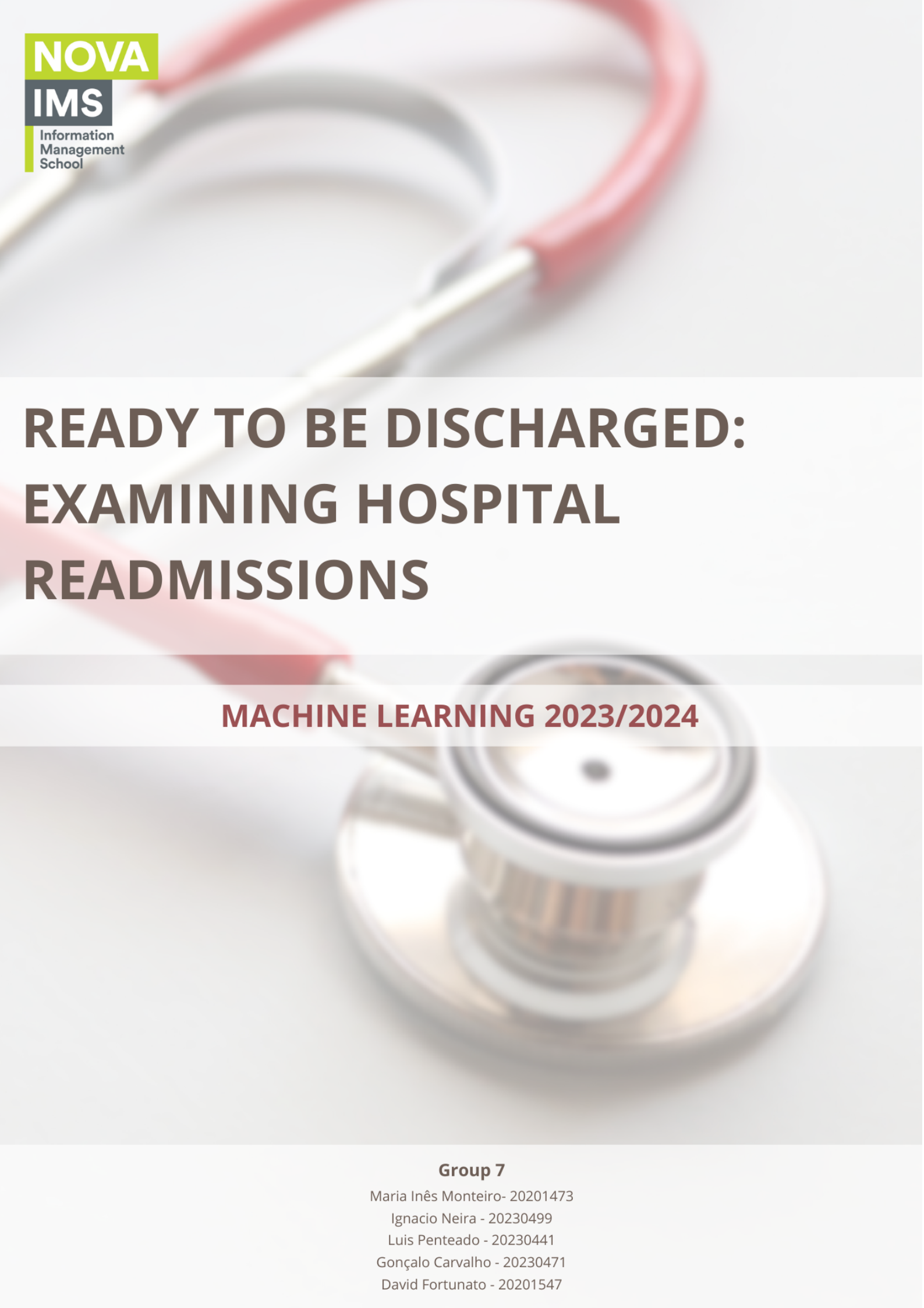


A close-up, slightly blurred image of a red stethoscope with a silver chest piece, resting on a light-colored surface. The stethoscope is positioned diagonally across the frame, with the chest piece in the lower right and the tubing curving towards the upper left.

NOVA

IMS

Information
Management
School

READY TO BE DISCHARGED: EXAMINING HOSPITAL READMISSIONS

MACHINE LEARNING 2023/2024

Group 7

Maria Inês Monteiro- 20201473

Ignacio Neira - 20230499

Luis Penteado - 20230441

Gonçalo Carvalho - 20230471

David Fortunato - 20201547

Abstract

In the realm of healthcare, improving patient care quality and managing healthcare costs remains one of the pivotal goals. This project focuses on developing predictive models to accomplish the following goals: binary classification, for predicting if a patient will be readmitted within 30 days, and multiclass classification, to forecast the specific timeframe of readmission as "No," "Readmitted in <30 days," or "Readmitted in >30 days." These models aim to empower healthcare providers with tools for timely intervention and adequate post-discharge care. Our main hypothesis suggests that utilizing machine learning models can accurately predict hospital readmissions for patients (primarily diabetic), offering actionable insights to implement targeted interventions and optimize post-discharge care. In this project, we resorted to several steps in order to obtain our predictions. These steps include data exploration and treatment, including missing values, outliers, and labeling; feature selection techniques; and applying, tuning and combining different models and methods. These steps were crucial to accurately evaluate our work over various important metrics, namely precision, recall, and f1. The results showed that, for the binary classification, the top-performing models were Random Forest, AdaBoost and Gradient Boosting Classifier. On the other hand, the top performing models for the multiclass were Decision Trees, Gradient Boosting Classifier and Neural Networks. However, after applying the Stacking method, we faced a significant improvement in the binary classification, by stacking Gradient Boost and AdaBoost. The final f1 scores were 0.345 and 0.646, for binary and multiclass, respectively. These results unveiled interesting insights not only about machine learning, but about hospital data traits. We discovered the power that Ensemble holds, the common issue that is the imbalanced characteristic of the datasets, and the difficulty in gathering and engineering relevant features to obtain satisfying results, what can injure healthcare quality and sustainable spending.

Index

Introduction	3
Data Exploration	3
Data Preprocessing.....	4
Missing values	5
Outliers	5
Feature engineering	5
Scaling.....	6
Classification	6
Binary Classification	7
Feature Selection	7
Results	8
Multiclass Classification	9
Additional Pre-Processing and Initial Steps.....	9
Feature Selection	9
Results	10
Conclusion.....	11
References	13
Annexes	15

Introduction

The issue of hospital readmissions, especially among diabetic patients, constitutes a complex challenge, which affects care quality benchmarks and has significant financial implications. Our project comprises of an exploration into this complex field, aiming to take advantage of the potential of machine learning methodologies and techniques to address the critical concern of hospital readmissions. Our primary focus revolves around constructing robust classification models designed to predict both the likelihood ("Readmission" or "No Readmission") and a more precise timeframe ("No Readmission", "Readmission in <30 days", and "Readmission in >30 days") of hospital readmissions after patient discharge.

The approach for this project aligns with the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology, encompassing the entire machine learning workflow. Starting from data exploration, preprocessing, and feature engineering, to feature selection and model optimization, aiming for a seamless deployment process. It is our goal not only to apply diverse machine learning approaches, such as Logistic Regression, Neural Networks, Decision Trees and Ensemble methods, developing predictive models with the capacity to classify the likelihood and timeframe of readmissions as good as possible; it is also our goal, in the whole process, to resource to pertinent concepts such as, for instance, encoding, train/test splits, cross-validation, confusion matrixes and hyperparameter tuning.

Similarly to this project, primarily in the binary class portion, other researchers have delved into predicting readmissions among diabetic patients, dealing with data such as demographics, clinical history, and laboratory results. These studies have revealed significant associations between patient characteristics, comorbidities, medication adherence, and the likelihood of readmission. In these researches, we have found that class imbalance was a significant concern, as it is a reoccurring theme in hospital readmission datasets. This may happen since hospital information is rather delicate and protected, resulting in datasets with reduced quality. This problem is the main reason why precision, recall and f1 scores are rarely high in these situations. Therefore, we expected to have similar issues and suboptimal results, in the mentioned metrics, by the end of our analysis.

Data Exploration

Data exploration is crucial in the beginning of any machine learning project, as it unveils insights into the structure, patterns, and quality of a dataset, allowing for informed decisions regarding data preprocessing, feature selection, and later, modeling.

The dataset provided, "train.csv", contained 71236 rows, the observations, and 31 columns, the features. Out of these 31 columns, 2 of these were our dependent variables (our target) the other 29 were our independent variables, which we would later use to predict our targets.

Right ahead, it was clear that our dataset was imbalanced, as the majority (88.8%) of the values of our target variable "readmitted_binary" were "No", meaning that the patient would not be readmitted to the hospital in 30 days. For "readmitted_multiclass", the majority of values correspond to "No" (53.91%), followed by ">30 days" (34.92%) and "<30 days" (11.16%).

One of the first things that we wanted to look for was a variable that could serve as our index, with "encounter_id" being the ideal candidate, as it contained only unique values and it makes sense to predict hospital readmission based on a certain encounter.

We also noticed that most of our features (18), were categorical which, from the start, implied a more complex preprocessing and a problem for some of the models that we would later apply. We checked descriptive statistics for both numerical variables and categorical variables to look for a preliminary understanding of the dataset's characteristics, such as central tendencies, variability, distributions, missing values, incoherencies, and potential outliers. It stood out that the variable "country" only had 1 unique value.

On the categorical variables, we noticed that some of them contained null values. To look further into the missing values issue, we decided to perform "value_counts" for every variable. The insight that we got was that there were variables that not only had null values, but also values that meant that the information in a specific observation was missing - values such as "Unknown", "Not Mapped", "Not Available" and "?", that would later need to be treated as well. It is also important to notice that variables such as "a1c_test_result" and "glucose_test_result" had a very high percentage of missing values, which normally would be a reason to drop them immediately. However, we thought that these missing values actually had a relevant meaning - that the patient did not take these tests; something similar also occurred with the variables "secondary_diagnosis", "additional_diagnosis" and "payer_code" - so these cases were also an option to have in consideration while exploring the data. Nonetheless, missing values with no apparent meaning also existed: this was the case with "weight", which had 96.95% null values, and "medical_specialty", which had 49.02% null values.

When exploring categorical variables, it was apparent that most of them had a lot of different values that had the potential of being aggregated in some way, which would make further encoding much simpler. For instance, discharge_disposition (25), "primary_diagnosis" (687), "secondary_diagnosis" (699), "additional_diagnosis" (747), "payer_code" (18), etc.

We noticed that 3 of the categorical variables ("primary_diagnosis", "secondary_diagnosis" and "additional_diagnosis") had the same concept: ICD-9 codes. This indicated that some valuable insights could possibly be taken from these codes.

Furthermore, our group decided to plot some visualizations to look for potential insights, patterns, and relationships within the data. We plotted histograms [Figure 1] to look at the frequency distribution of the numerical variables: only 4 variables seemingly normally distributed such as "age" and "number_lab_tests", so we need to take that into consideration for future feature selection and modeling results. We created box plots [Figure 2] to look for outliers in numerical data, in which we saw that 11 variables had outliers, which would need further treatment. This helped us identify the variables in which we need to be more careful in the preprocessing step. Lastly, we plotted pairwise relationships of the variables [Figure 3], in which we detected no significant relationship insights, and confirmed the abundance of binary variables.

When it comes to incoherencies, we did not find anything more that would need to be preprocessed, neither did we find duplicated rows that would need to be deleted.

Data Preprocessing

Based on the previous exploration process, our group had to take some steps in order to clean, transform, and organize the dataset, ensuring optimal model performance.

Missing values

Missing values refer to the absence of data in specific variables or observations within a dataset, which can impact analysis and further model performance. Having this into account, we decided to take several personalized approaches for this problem.

Aggregating unknown values - As mentioned in the exploration chapter, some missing values were presented under the labels “Unknown”, “Not Mapped”, “Not Available” or “?”, on top of null (NaN) values. This is what happened with the variables “race”, “admission_type”, “admission_source” and “discharge_disposition”. So, in order to avoid redundancies in feature engineering, we aggregated these kinds of values in each variable into a single category. In other words, we wanted to have, in each variable, only one type of value to represent unknown values.

Missing values with meaning - The variables which had missing/unknown values that could actually be meaningful to our model were “glucose_test_result”, “a1c_test_result”, “secondary_diagnosis”, “additional_diagnosis” and “payer code”. So, we just replaced the missing values with an adequate label for its meaning, such as “didn’t take” [that test].

Replacing with the mode - The variable “gender” had about 3 Invalid/Unknown values. As these were a very little portion of all the observations, we just replaced them with the mode of the variable.

K-Nearest Neighbors - The variable age had about 5% missing values and we saw this as a good opportunity to apply the K-nearest Neighbours method, using the default number of neighbors, 5. It is important to note that this method was applied only after turning this variable into a numerical one and after the scaling of the data, as we will discuss later.

No treatment - The variables weight and medical_specialty were not treated, as according to what we discussed in the exploration chapter, they were planned to be dropped in feature selection.

Outliers

To identify the outliers of each numerical variable, we used the interquartile range (IQR) method. As a result, we identified 11 variables with outliers. In the case of “age”, “length_of_stay_in_hospital”, “non_lab_procedures”, “Charlson_Cormobility_Index” and “encounter_count” (in the case of the last two, new engineered features which will be explained below) we decided not to treat them because the outlier values appear to be “real data” without such exaggerated values. For “outpatient_visits_in_previous_year”, “emergency_visits_in_previous_year” and “inpatient_visits_in_previous_year” we decided to limit each to the highest value in 10, as this was a reasonable limit, and we assumed that after that bound, all values reflected the same behavior. For the last three variables (“number_of_lab_tests”, “number_of_medications” and “number_of_diagnoses”) we used the upper bound given by the IQR analysis to cap the highest value for each one and we assumed that after that bound all values reflected the same behavior.

Feature engineering

The goal of this step was to enhance model performance by creating new and modifying existing features from the dataset provided. We decided to take different approaches for different variables based on what we needed to take from them.

As explained before, many categorical variables had many different values - which had the potential of being grouped into fewer categories. This is what happened with variables such as "discharge_disposition", "glucose_test_result", "a1c_test_result" and "medication". The goal was to prevent overfitting and reduce complexity in the model while capturing essential patterns and improving generalization performance. There is also the particular case of "age", as we replaced the age intervals by the mid-point of that interval, turning it into a numerical variable.

Binary variables - To deal with variables, such as "gender", "payer_code", "change_in_meds_during_hospitalization", "prescribed_diabetes_meds" and even our target variables, we turned them into dummy variables (only have "1's" and "0's" as values meaning, respectively, true and false), facilitating numerical representation.

Target encoding - One of the main problems we faced in this project was the abundance of categorical variables. We decided to face this issue using target encoding (for both the binary and multiclass targets), applying it to every categorical variable that was not turned into a dummy variable. This concept is further explained in the Annex.

Creation of new variables - Having in mind the 3 variables that had the concept of ICD-9 codes, our group decided to create a function to assign a score to each diagnosis code and added them into a final value. This new calculated column is called "Charlson Comorbidity Index". This concept is further explored in the Annex. Additionally, we created a variable called "encounter_count" by counting how many times each patient had an encounter in the hospital (counting the number of times the same "patient_id" appeared in the dataset).

Scaling

Before moving on to the modeling step of the project, it was crucial to scale our data to ensure that features with different scales/units have a comparable impact on our model. This prevents bias towards variables with larger magnitudes. We decided to use Robust Scaler to achieve this, as this scaling method preserves data distribution and is less sensitive to outliers compared to Min-Max Scaler. Although we treated some outliers in our models, we still had some extreme values.

Classification

Before moving on to the modeling step of our project, we wanted to assure what our priorities were when it came to the metrics tested. We highlighted the importance of recall and precision. If the recall is too low, it means that the hospital might not be counting with the necessary resources that it should have to receive readmitted patients - reputation of the hospital might be hurt because the hospital is assuring the patients, they will not need further treatment. If the precision is too low, the hospital might be counting on receiving a lot more patients (therefore having more costs) than it will actually have, resulting in a waste of money/resources. However, the metric `f1_score` was our priority metric, as it works as a balance between precision and recall.

Binary Classification

Feature Selection

After data exploration and preprocessing, there is a crucial step while working in a machine learning model. Feature selection identifies the most relevant and informative features from our dataset, which will consequently be selected in order to improve model performance, reduce overfitting, enhance computational efficiency, and create simpler, more interpretable models. To achieve this, we performed several techniques that, combined, provided us the optimal set of features to use in the modeling process.

Irrelevant Variables - In the exploration step of this project, our group noticed that there were some variables that, right away, showed that they were of no good use for our model. There were variables that had mostly missing values, such as “weight” (97%) and “medical_specialty” (49%). As not removing them could potentially cause bias in our model due to imputation, and we did not consider them crucial for our analysis, we decided to drop them. We also had a variable with no variability, “country” - as it showed no discriminative power, and just added complexity to our model, we also dropped it.

Point Biserial Correlation - Next, we checked for high correlations between our variables, as it could indicate redundancy or multicollinearity and potentially impact our model's stability and interpretability. As we were dealing with both binary and numerical variables, we compared these correlations through a Point Biserial Correlation heatmap [Figure 4], which showed us that there were two variables which had high correlation, 0.81 – “medication” and “prescribed_diabetes_meds”. As medication had higher correlation with the remaining variables overall, we decided that it was the less relevant variable between the two.

Lasso - We also performed LassoCV (Lasso with Cross-Validation) with the intent of encouraging sparsity and shrinking the variables’ coefficients to zero, consequently eliminating less important (less informative) variables. The ones that fit this criterion were “a1c_test_result” and “glucose_test_result”.

Mutual Info Classification - Mutual info classification measures the dependency between features and the target variable, enabling the selection of the most informative features for classification tasks. To make our decision of which features to keep, we analyzed the bar chart [Figure 5] and classified as not relevant the features from which there was a significant drop in importance. The ones that fit this criterion were “average_pulse_bpm”, “number_lab_tests”, “non_lab_procedures”, “number_of_medications”, “a1c_test_result” and “glucose_test_result”.

ANOVA - Similarly to Mutual Info Classification, we performed ANOVA, as it assesses the statistical significance of the variables. By looking at the chart [Figure 6], we decided that the variables that had a significant drop on their impact on the target variable were “race”, “gender”, “payer_code”, “age”, “admission_type”, “average_pulse_bpm”, “number_lab_tests”, “non_lab_procedures”, “a1c_test_result”, “glucose_test_result” and “change_in_meds_during_hospitalization”.

Random Forests - Lastly, we performed Random Forests as a feature selection method as it assesses the importance of each variable in predicting the target. By looking at the chart [Figure 7] we decided that “glucose_test_result” and “prescribed_diabetes_meds”, compared with the rest of the variables, were not relevant for target prediction.

Conclusion - Considering all the 5 methods described above, our group created a final table [Table 1] which showed us, overall, the most important features to retain, and which ones to drop - “average_pulse_bpm”, “number_lab_tests”, “non_lab_procedures”, “glucose_test_result” and “a1c_test_result”. We decided to use 60% relevancy as a threshold to keep a variable.

Results

Following feature selection and identification of the optimal set of variables, we moved on to exploring diverse models and methodologies to pinpoint the most suitable solution for our problem.

We opted by accessing our model through the holdout method, splitting the data into a train subset (70%) and a validation (application of the model in unseen data) subset (30%).

We started by selecting a wide range of models to try, as lectured in the practical classes, such as Linear Regression, Instance Based Learning (KNN), Gaussian Naïve Bayes, Neural Networks, Decision Trees, Random Forests, AdaBoost Classifier, Gradient Boosting Classifier and Support Vector Machines.

Imbalanced Data Issue - As mentioned before, one of the main problems we found in data exploration was the imbalanced quality of the dataset. Due to this, the precision, recall and f1 scores were extremely low in every model mentioned above. Our group decided to experiment techniques that could possibly repair this problem by reducing or replicating/synthesizing instances of the dataset – Undersampling and Oversampling (SMOTE), as it can be found further explained in the annexes. We applied these 2 techniques in different ways (individually and collectively) and found significantly better results regarding the f1 score.

Disregarding models - After checking the results in every model, we came to some conclusions on which models would not be ideal as our final solution. KNN and Gaussian Naïve Bayes returned significantly lower f1 scores than the rest of the models. Neural Networks also did not return optimal scores, which was an interesting insight, as we thought that by dealing with non-linear data the results would be better; we concluded that the reason might be because neural NN works better with a very large amount of data, when our scenario only had 70 000 observations.

Overfitting - Even after applying undersampling and oversampling, we still had an issue with overfitting, as the f1 scores for training data were significantly higher than the f1 scores for validation data. Therefore, we decided to take a different approach – the parameter “class_weight” and “sample_weight”, assigning a much larger importance (weight) to the minority class “1” and less importance to the majority class “0”. This solution not only fixed the imbalanced data issue, but also fixed the overfitting issue. More explanation on these parameters can be found in the annex.

Grid Search - Without overfitting and not-so-low scores, our group wanted to find the optimal potential (parameters) of the remaining models/methods, by applying GridSearchCV, which can be found further explained in the annex. We performed this several times for each model, always trying to converge the parameter sets into the most optimal, using the results from each previous search. We also experimented with some manual tuning after applying Grid Search, as it was very time consuming.

After Grid Search, we came to some conclusions:

Decision Trees: While it returned good results (0.32 f1_score), we disregarded it as it is prone to overfitting, may lack generalization, and it has, in general, less robustness and predictive power comparing with other tree-based models and learning methods, Ensemble methods.

Support Vector Machines: With SVM, the results were good (0.32 f1_score). However, we disregarded it because this model is very computationally expensive compared to the others.

Logistic Regression: We disregarded LR because as it assumes a linear relationship, it may not be enough to fairly describe our data, even though its f1_score was, overall, good (0.31 f1_score).

Additionally, all these 3 dropped models also appeared to have a much lower Recall scores than the remaining ones, which was also a contributing factor for their disregardance.

Having this in mind, we selected our final 3 options: Random Forests, Adaboost and Gradient Boost, which had good f1 scores (0.32, 0.32 and 0.33, respectively) and the tradeoffs between precision and recall were very similar. Therefore, we nominated Gradient Boost as our best “standalone” method.

Stacking - After identifying Gradient Boost as the top-performing model, we aimed to explore ways to enhance its performance by combining it with our other best models, AdaBoost and Random Forests, through stacking. Initially, we also experimented stacking the three models together, but we opted against this approach as it returned suboptimal f1 scores, potentially due to the noise introduced by combining three models.

However, after trying both model combinations, we observed that stacking AdaBoost and Gradient Boost produced the highest f1 score (0.345) for our validation dataset. It had similar precision, but slightly lower recall values (0.23 and 0.67, respectively) and no signs of overfitting. Finally, we concluded that stacking AdaBoost and Gradient Boost would serve as our final model to be tested in the “test” data.

A final table containing all results can be found in [Table 2].

Multiclass Classification

Additional Pre-Processing and Initial Steps

For the multiclass classification, our group decided to use the same steps (just changing the target variable) as for the binary classification. Regarding the treatment of our target variable, we applied a label encoding method to transform it into a numerical variable with three options: 0 (“<30 Days”), 1 (“>30 Days”) and 2 (“No”).

Feature Selection

To reduce overfitting, increase computational efficiency and produce a simpler and more interpretable model, we applied the same techniques as for binary classification, but with the multi-class target variable, meaning that we obtained some different results in Lasso, Mutual Info Classification, ANOVA and Random Forest.

Irrelevant Variables - Just like in Binary classification, we started by dropping the variables which showed to be irrelevant to our model – “weight”, medical_specialty and “country”.

Point Biserial Correlation - We proceeded by analyzing the Point Biserial Correlation heatmap [Figure 8] for the multiclass, which showed us a very high correlation between the same two variables which had a high correlation in the binary classification, “medication” and “prescribed_diabetes_meds”. We came to the same conclusion, disregarding “medication”.

Lasso - In LassoCV, we aimed for dropping the 2 variables with coefficients equal to zero. However, this time we noticed that “gender” had a coefficient very close to zero. By applying a threshold of 0.0015, we disregarded, “glucose_test_result”, “a1c_test_result”, and “gender”.

Mutual Info Classification - for Mutual Info Classification [Figure 9], we decided to analyze its significance bar chart and drop the variables that looked significantly less important in comparison with the remaining ones – “age”, “admission_type”, “non_lab_procedures”, “glucose_test_result”, “a1c_test_result”, “change_in_meds_during_hospitalization” and “prescribed_diabetes_meds”.

ANOVA – For ANOVA [Figure 10], we proceeded with the same method as Mutual Info Classification, disregarding “race”, “gender”, “age”, “payer_code”, “average_pulse_bpm”, “number_lab_tests”, “non_lab_procedures”, “glucose_test_result”, “a1c_test_result”, and “Charlson_Cormobidity_Index”.

Random Forests - For Random Forests [Figure 11], we proceeded with the same method as Mutual Info Classification and ANOVA, disregarding “glucose_test_result” and “prescribed_diabetes_meds”.

Conclusion - With the 5 previous methods, we decided to realize a majority vote [Table 3] to find out which variables to drop and which to keep (over 60% of relevance to keep a variable). In summary, we decided to drop 6 variables: “gender”, “age”, “non_lab_procedures”, “glucose_test_result”, “a1c_test_result” and “prescribed_diabetes_meds”.

Results

We reproduced the modelling steps as seen for the binary class. This means we used the same models and the metric f1_score was our priority metric.

Since we are working with multiclass classification, it is mandatory to establish a value for the hyperparameter ‘average’ in f1_score function. This way the only options are ‘micro’, ‘macro’ and ‘weighted’. We used “f1_score weighted” because this measure takes into account the label imbalance and, as we have an unbalanced dataset, we want to represent the minority class as well as possible.

Imbalanced Data Issue - Using the holdout method, we split the data into a train subset (70%) and a validation subset (30%). For the models we used the same models as in the binary problem like Linear Regression, Instance Based Learning (KNN), Gaussian Naïve Bayes, Neural Networks, Decision Trees, Random Forests, AdaBoost Classifier and Gradient Boosting Classifier.

Unlike the binary classification, we did not find a need to use the parameter “class_weight” or “sample_weight” in the models since as we mentioned before is “less imbalanced” compared

with the binary model, giving more information to the models (more classes) to learn, and forcing models to learn from a greater number of relationships.

Even with the above mentioned, the dataset was still imbalanced for the minority class (11%) and this is the main reason for us to use the `f1_score` weighted to evaluate the models.

Disregarding models - We started by running all the models with the hyper-parameters all set by default. Comparing the results for each model [Table 4], we have excluded two models: Gaussian Naive and KNN, as they returned a validation score below 0.6, which was worse than the remaining models. These results make sense, as KNN does not work well with high dimensionality and Gaussian Naive has limitations in capturing complex patterns.

In this step, even though the validation score of the Decision Tree model was below 0.6, we kept this model, as the Random Forest returned a better result and we believed that there was a chance of optimizing Decision Trees.

Grid search - Similarly to the binary classification, we ran a GridSearchCV several times for the rest of the models. We selected the best results and then compared them. The only exception for the GridSearchCV was the Neural Networks, as the process required a high computational cost, and it was not possible to run the process to the end.

After comparing the results, we choose the Gradient Boost as our best model based on the `f1_score` (quite close to Neural Networks') and having better precision and recall; additionally, Gradient Boost runs faster than Neural Networks. The second and the third one was Neural Networks and Decision Trees, respectively. We disregarded the Random Forest since it showed overfitting.

Decision Trees surprised us by returning a significantly improved result [Table 5], outperforming Random Forest. Additionally, Neural Networks also gave a good result compared to binary classification.

Stacking - After identifying Gradient Boost as the top-performing model, we aimed to explore ways to enhance its performance by combining it with our other best models: Neural Network and Decision Trees. After trying the following combinations: "Gradient Boost, Neural Network and Decision Tree" and "Gradient Boost and Neural Networks", the best result between them was the second one ("Gradient Boost and Neural Networks"), probably due to the noise the 3 stacked models produced. The result comparing with Gradient Boost itself was basically the same (0.64), while spending more time.

Considering everything mentioned above, we concluded that Gradient Boosting Classifier served as our final model to be tested in the "test" data.

Conclusion

Having finished the exploration, preprocessing, feature selection and modelling steps of our project, we can make some conclusions about our findings. As mentioned above, precision, recall, and `f1` scores were our priority metrics in the making of our model, and we can verify that the scores were, overall, low, especially when it comes to recall and `f1`. This means that the final model has difficulty in correctly identifying patients who are at risk of being readmitted to the hospital and, in general, difficulty in classifying whether a patient is prone to being readmitted or not. However, the multiclass model performed a lot better, which means that the model has

more ease in identifying whether a patient will be readmitted in more or less than 30 days, or not readmitted at all, which is a more helpful way of achieving the main goal of improving patient care quality and hospital cost savings.

Some of our findings clashed with our initial expectations, regarding either dataset characteristics, feature selection or modeling. For instance, there were variables in the dataset which we initially thought would be irrelevant (such as payer code and patient_id) but turned out to be some of the most important and improved features in our model. Additionally, the differences between the performance of the multiclass and binary classifications were initially unexpected; however, we discovered that the multiclass feature was much less imbalanced, as the new class gave the models more information to learn from, forcing the models to learn from a greater number of relationships, hence the better results. Furthermore, the best performing methods that we found were all ensemble methods – Random Forests, AdaBoost and Gradient Boost, which matched our initial expectation, as these methods typically have the ability to combine weaker models into a more robust predictor, handling diverse patterns, insights, and relationships from the dataset. We also expected that, by stacking our best models, the overall predictive performance would be enhanced, by capturing a wider range of patterns in data. Moreover, we also did not expect Logistic Regression to perform as good as it did (in terms of f1_score), as it was the simplest model used. Lastly, in the multiclass classification, we were surprised by the performance of Decision Trees, since the model returned very good results despite being less computationally intensive compared to the ensemble models. For that reason, it could be a good proposition for use with larger datasets.

Regarding limitations of our work, there were techniques that we would have liked to experiment (K-fold cross validation as a model assessment technique, for instance) but did not due to lack of time to implement them correctly. We would like to have applied more techniques in feature selection, so we could get a more complete overview of the most relevant features to keep. Due to the time constraint, we were also not able to experiment with more variable transformations and variable creations in data preprocessing, as well as different ways to deal with missing values (for instance, KNN input for some categorical variables). Another limitation was the fact that we could not use more robust models, outside scikit-learn, such as XGBoost. Finally, we would have liked to try other parameter tuning in the models in order to find the most optimal parameter set for each model, which we did not due to the lack of time and resources.

After finishing this project and analyzing the suboptimal results as mentioned above (mainly in binary class), we recognize potential for improvement. We suggest the hospital to establish more strict procedures to collect data, as well as get field experts to study what can be important characteristics of a hospital-readmitted patient. A good dataset is fundamental for a good model. However, it is understandable that, as we are dealing with such a sensitive domain like healthcare, security measures need to stay in place, ensuring data privacy. Finally, as in any model, we suggest continuous monitoring and maintenance of the performance.

References

Charlson Comorbidities - Coding Algorithms for ICD-9-CM and ICD-10.pdf. (n.d.). Manitoba Centre for Health Policy. <http://mchp-appserv.cpe.umanitoba.ca/concept/Charlson%20Comorbidities%20-%20Coding%20Algorithms%20for%20ICD-9-CM%20and%20ICD-10.pdf>

Charlson Comorbidity Index Score. (n.d.). ResearchGate. https://www.researchgate.net/figure/CCI-Charlson-comorbidity-index-score_fig3_301936501

de la Fuente, J., & García, J. (2022). Development and validation of a predictive model for early unplanned readmission in a general hospital in Spain: A multicentre study. BMC Health Services Research, 22(1), 62. <https://doi.org/10.1186/s12913-022-08748-y>

Dimitriou, C., & Gradel, M. (2019). [The Danish Charlson Comorbidity Index]. Ugeskrift for Laeger, 181(34). PMID: 31428236. <https://pubmed.ncbi.nlm.nih.gov/31428236/>

GridSearchCV. (n.d.). Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

GridSearchCV: Hyperparameter Tuning in Machine Learning. (2022). Great Learning. <https://www.mygreatlearning.com/blog/gridsearchcv/>

LogisticRegression. (n.d.). Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

KNeighborsClassifier. (n.d.). Scikit-learn. <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>

MLPClassifier. (n.d.). Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

GaussianNB. (n.d.). Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.GaussianNB.html

SVC. (n.d.). Scikit-learn. <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>

DecisionTreeClassifier. (n.d.). Scikit-learn. <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>

RandomForestClassifier. (n.d.). Scikit-learn. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

AdaBoostClassifier. (n.d.). Scikit-learn. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html>

GradientBoostingClassifier. (n.d.). Scikit-learn. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html>

StackingClassifier. (n.d.). Scikit-learn. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.StackingClassifier.html>

Koushik, V. (2021). Grid Search for Model Tuning. Towards Data Science. <https://towardsdatascience.com/grid-search-for-model-tuning-3319b259367e>

Koushik, V. (2021). Feature Selection Based on Mutual Information Gain for Classification and Regression. Towards Data Science. <https://guhanesvar.medium.com/feature-selection-based-on-mutual-information-gain-for-classification-and-regression-d0f86ea5262a>

Koushik, V. (2021). Using ANOVA for Feature Selection in Python. MachineHack. <https://machinehack.com/story/using-anova-for-feature-selection-in-python>

Rajeev, K. (2021). The Importance of Healthcare Data Security. Prime TSR. <https://primetsr.com/insights/the-importance-of-healthcare-data-security/#:~:text=Health%20information%20contains%20more%20private,and%20safety%20of%20healthcare%20organizations.>

Reddy, P. (2021). Why Go Large with Data for Deep Learning? Towards Data Science. <https://towardsdatascience.com/why-go-large-with-data-for-deep-learning-12eee16f708>

Reddy, P. (2020). Micro/Macro Weighted Averages of F1 Score Clearly Explained. Towards Data Science. <https://towardsdatascience.com/micro-macro-weighted-averages-of-f1-score-clearly-explained-b603420b292f>

f1_score. (n.d.). Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html

Annexes

Charlson Comorbidity Index

The Charlson Comorbidity Index, a concept used in the transformation of the variables `primary_diagnosis`, `secondary_diagnosis` and `additional_diagnosis`, is an index that predicts the mortality risk for patients based on their condition, by assigning weighted scores to various medical conditions, represented by ICD-9 codes. The full table with every comorbidity can be found in [Table 6]

Oversampling and Undersampling

As we were dealing with an imbalanced dataset, where the dominance of one class (0, in this case) can lead models to favor the majority class, impacting predictive performance, we had to take some measures to overcome this problem. Two options stood out to us: Undersampling and Oversampling. While Undersampling involves reducing the instances of the majority class, while oversampling replicates or synthesizes instances of the minority class, both are capable of enhancing the model's ability to recognize patterns and make accurate predictions across all classes. However, by trying both techniques separately and even joining them, we always faced overfitting problems.

Target Encoding

Target encoding is a technique where categorical variables are replaced with the mean of the target variable corresponding to each category, turning these variables into numerical data, capable of being manipulated in further modeling.

GridSearchCV

Grid Search is a method to find the optimal hyperparameter combination in a specified model. In this project, we used `GridSearchCV`, a scikit-learn library which performs hyperparameter tuning by exhaustively searching over a grid of parameter values (`param_grid`), employing cross-validation (`cv`) to determine the “best combination”. In this case, the “best combination” is the grid which returns the higher `f1_score` (scoring).

It can be applied in the following way:

```
GridSearchCV(model, param_grid, scoring='f1', cv=5)
```

`class_weight` / `sample_weight` Parameters

As discussed before, throughout the implementation of the model, our group faced some problems with overfitting. One of the methods that we tried to fix this problem was the use of the `class_weight` and `sample_weight` parameters in each model.

The `class_weight` parameter assigns different weights to classes within the dataset, allowing the model to account for class imbalance, giving higher importance to underrepresented classes in the training data. It adjusts the impact of each class's contribution to the model, effectively influencing the learning process to prioritize better performance on minority classes. The `sample_weight` parameter has the same purpose as `class_weight`. It assigns different weights to

individual samples within the dataset, influencing the importance of each sample during training.

Grid Search Parameters (not lectured in class)

While applying Grid Search in our models, we experimented with some parameters which were not lectured in class, such as:

- “C”: Used in Logistic Regression and Support Vector Machines’s Grid Search, it represents a regularization parameter. Smaller values of C indicate stronger regularization.
- “penalty”: Used in Logistic Regression’s Grid Search, it specifies the type of regularization to be applied. The values taken can either be L1 (adds the absolute values of the coefficients to the cost function) or L2 (adds the squared magnitudes of coefficients to the cost function, penalizing large coefficients).
- “solver”: Used in Logistic Regression’s Grid Search, it refers to the optimization algorithm used to fit the logistic regression model. Different solvers (liblinear, lbfgs, newton-cg, sag, and saga) are suitable for different types of problems/datasets. For instance,, ‘liblinear’ is a good choice for smaller datasets, while ‘sag’ and ‘saga’ are better for large ones.

Figures

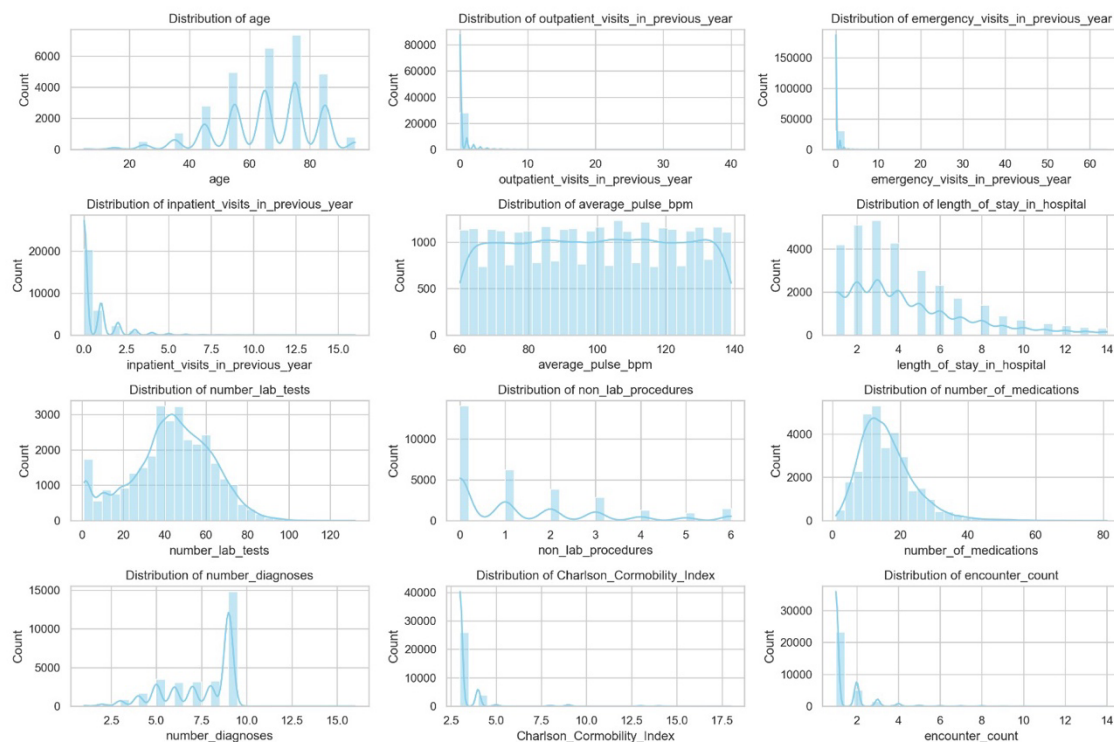


Figure 1 - Numerical Variables' Distributions

Numeric Variables' Box Plots

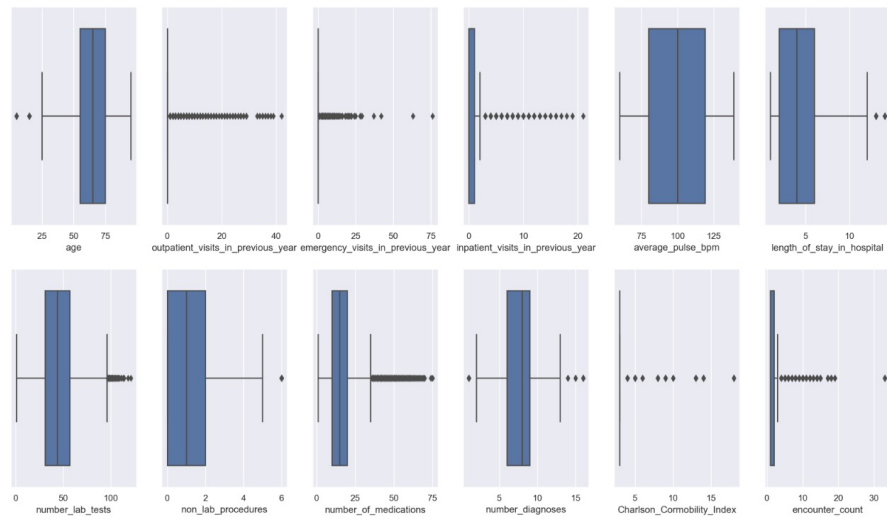


Figure 2 - Numerical Variables' Boxplots

Pairwise Relationship of Numerical Variables

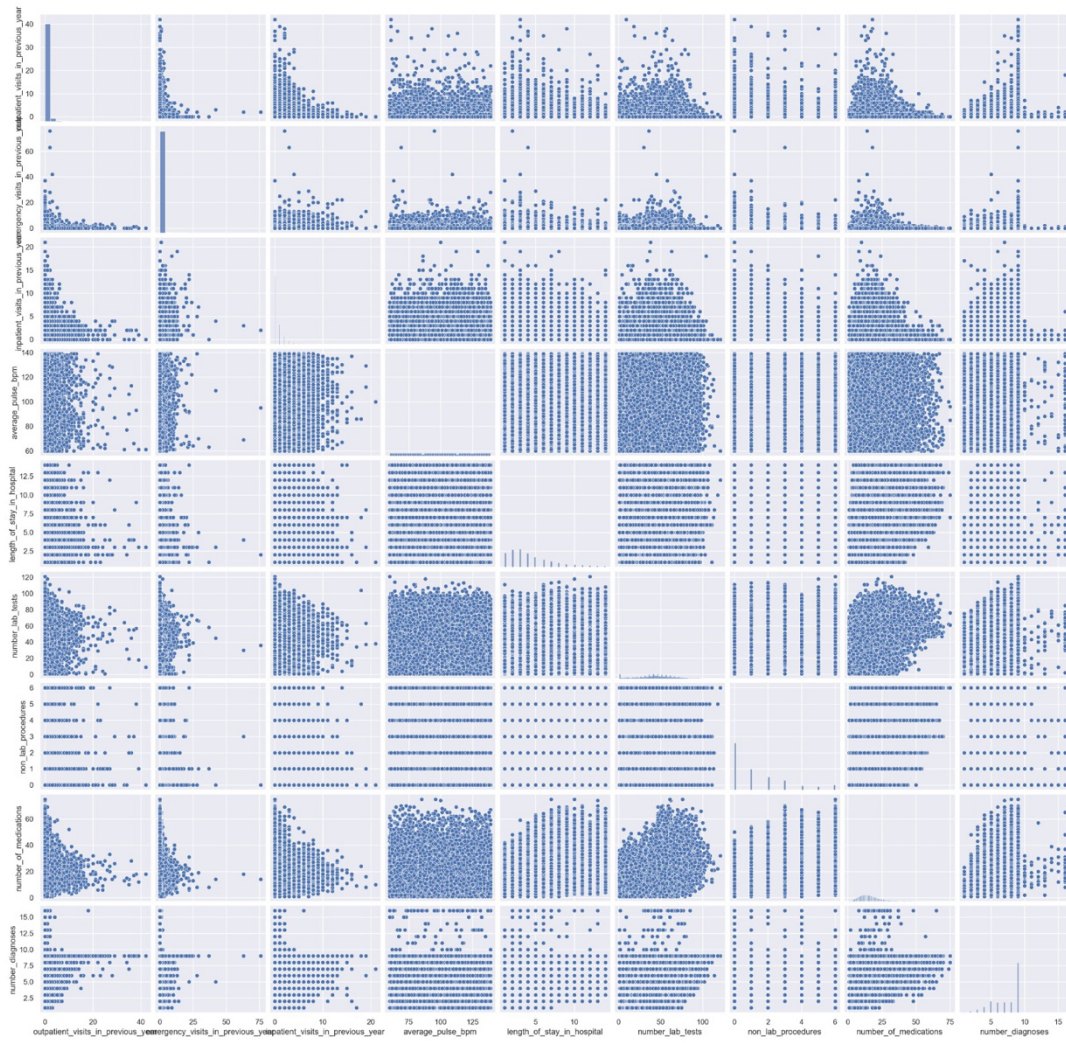


Figure 3 - Pairwise Relationships

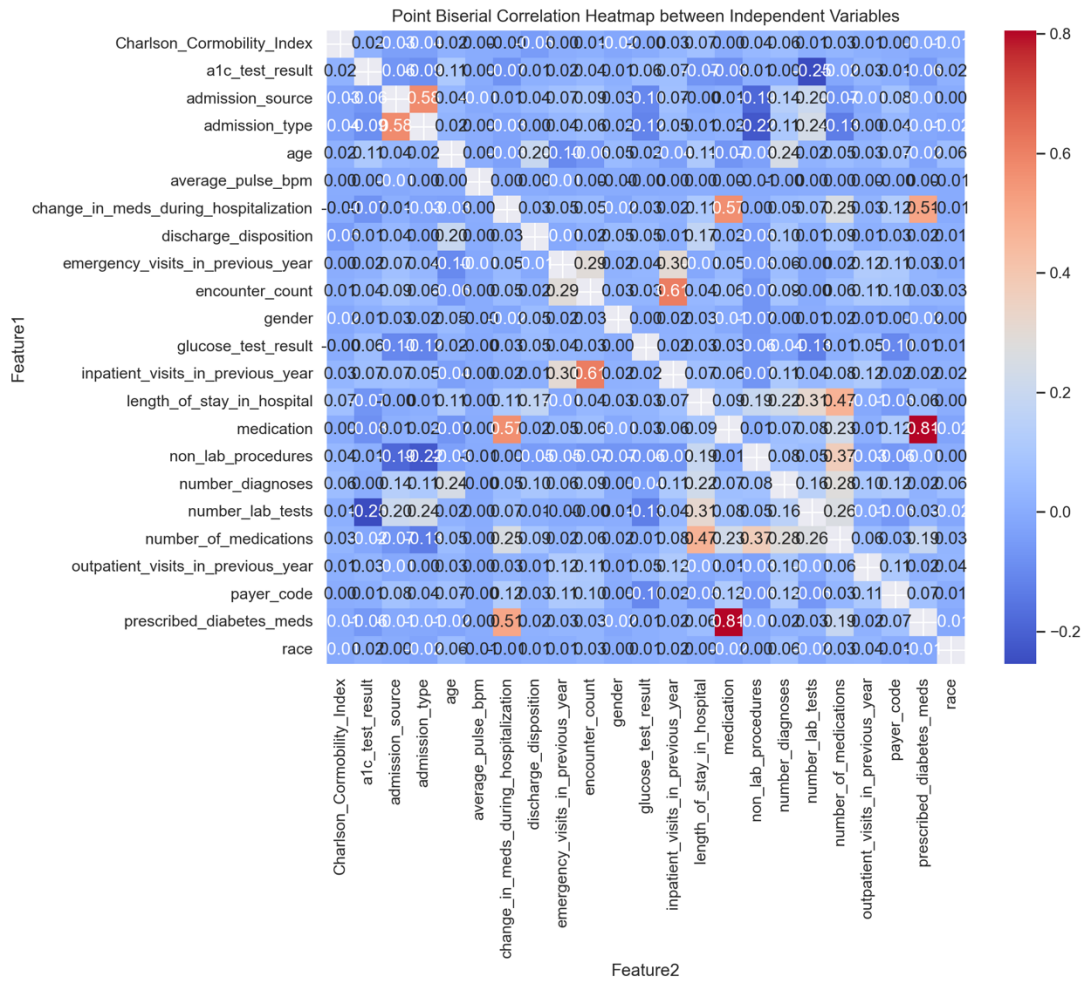


Figure 4 - Point Biserial Correlations - Binary Classification

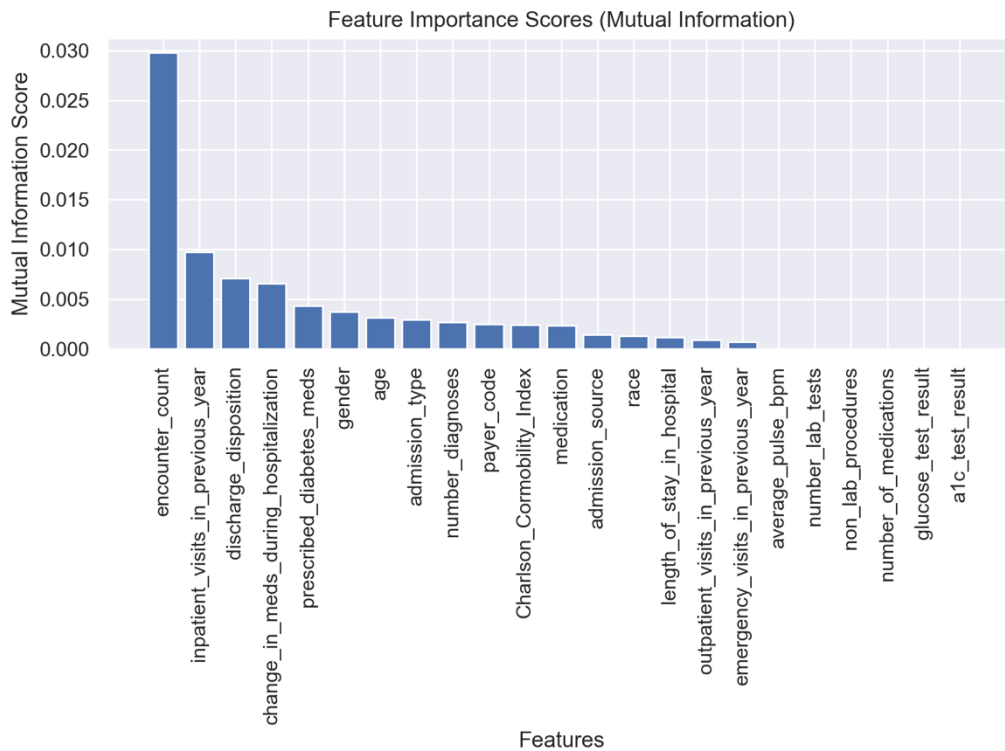


Figure 5 - Mutual Info Classification - Binary Classification

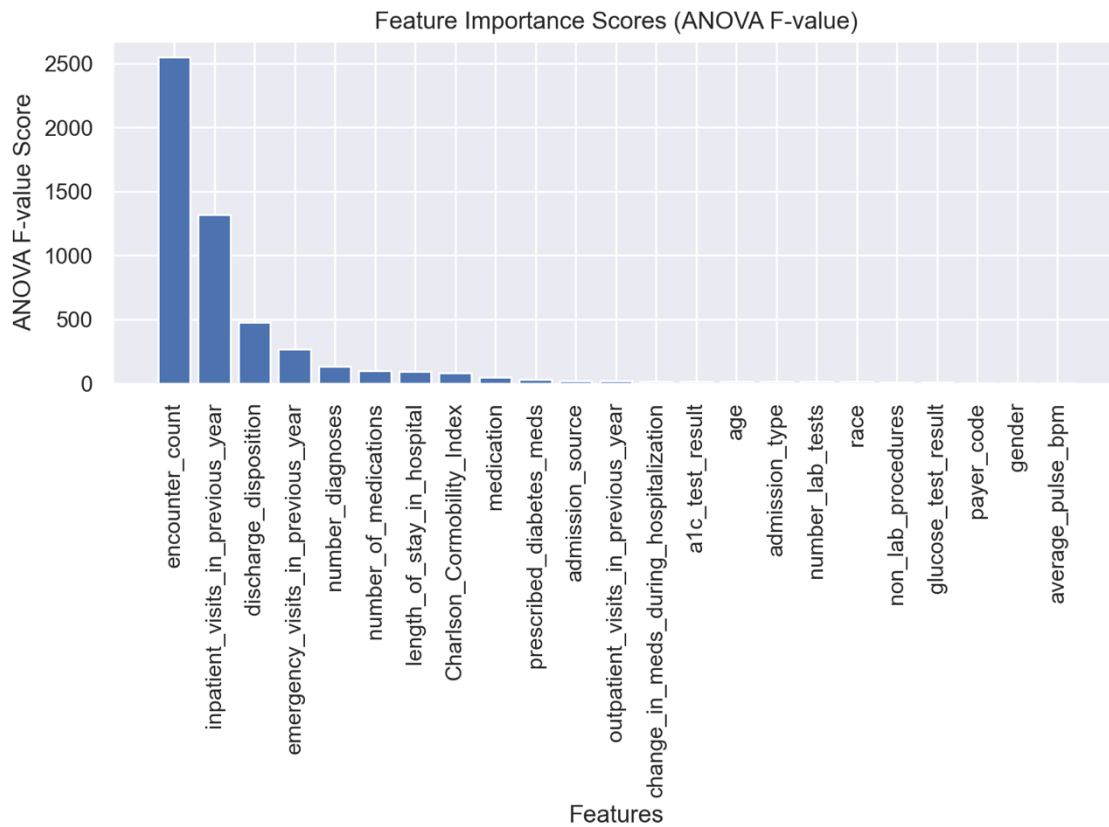


Figure 6 - ANOVA - Binary Classification

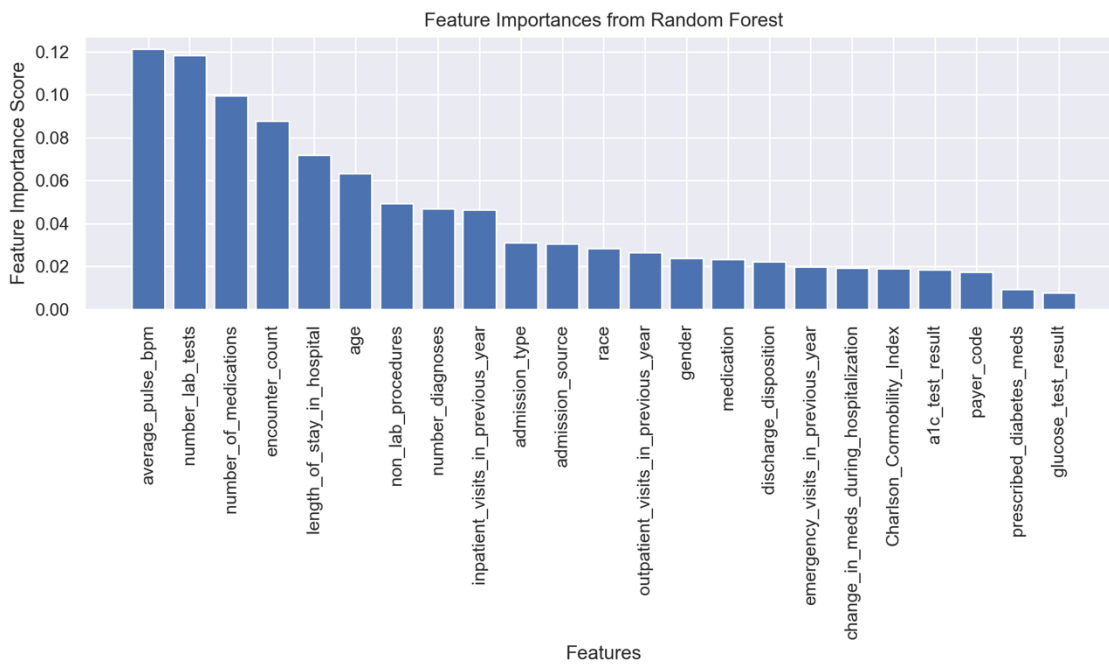


Figure 7 - Random Forests - Binary Classification

	Variable	Lasso	Point Biserial	Kbest - MIC	Kbest - ANOVA	Random Forests	N° True	%True	Decision
0	race	True	True	True	False	True	4	0.8	Keep
1	gender	True	True	True	False	True	4	0.8	Keep
2	age	True	True	True	False	True	4	0.8	Keep
3	payer_code	True	True	True	False	True	4	0.8	Keep
4	outpatient_visits_in_previous_year	True	True	True	True	True	5	1.0	Keep
5	emergency_visits_in_previous_year	True	True	True	True	True	5	1.0	Keep
6	inpatient_visits_in_previous_year	True	True	True	True	True	5	1.0	Keep
7	admission_type	True	True	True	False	True	4	0.8	Keep
8	average_pulse_bpm	True	True	False	False	True	3	0.6	Drop
9	discharge_disposition	True	True	True	True	True	5	1.0	Keep
10	admission_source	True	True	True	True	True	5	1.0	Keep
11	length_of_stay_in_hospital	True	True	True	True	True	5	1.0	Keep
12	number_lab_tests	True	True	False	False	True	3	0.6	Drop
13	non_lab_procedures	True	True	False	False	True	3	0.6	Drop
14	number_of_medications	True	True	False	True	True	4	0.8	Keep
15	number_diagnoses	True	True	True	True	True	5	1.0	Keep
16	glucose_test_result	False	True	False	False	False	1	0.2	Drop
17	a1c_test_result	False	True	False	False	True	2	0.4	Drop
18	change_in_meds_during_hospitalization	True	True	True	False	True	4	0.8	Keep
19	prescribed_diabetes_meds	True	True	True	True	False	4	0.8	Keep
20	medication	True	False	True	True	True	4	0.8	Keep
21	Charlson_Cormobility_Index	True	True	True	True	True	5	1.0	Keep
22	encounter_count	True	True	True	True	True	5	1.0	Keep

Table 1 - Feature Selection Summary - Binary Classification

	Model	Time	Train	Validatio	Precision	Recall
0	Logistic Regression	0.034	0.312	0.313	0.211	0.608
1	Gaussian Naive	0.044	0.218	0.220	0.257	0.193
2	KNN	1.238	0.996	0.039	0.408	0.021
3	Neural Network	0.041	0.160	0.100	0.379	0.058
4	Decision Tree	0.032	0.326	0.325	0.223	0.599
5	Random Forest	1.481	0.327	0.322	0.205	0.751
6	Adaboost	1.073	0.324	0.323	0.211	0.697
7	Gradient Boost	0.124	0.336	0.332	0.213	0.750
8	SVM	33.819	0.331	0.329	0.237	0.536
9	Stacking	1.258	0.353	0.346	0.233	0.671

Table 2 - Binary Classification Overview

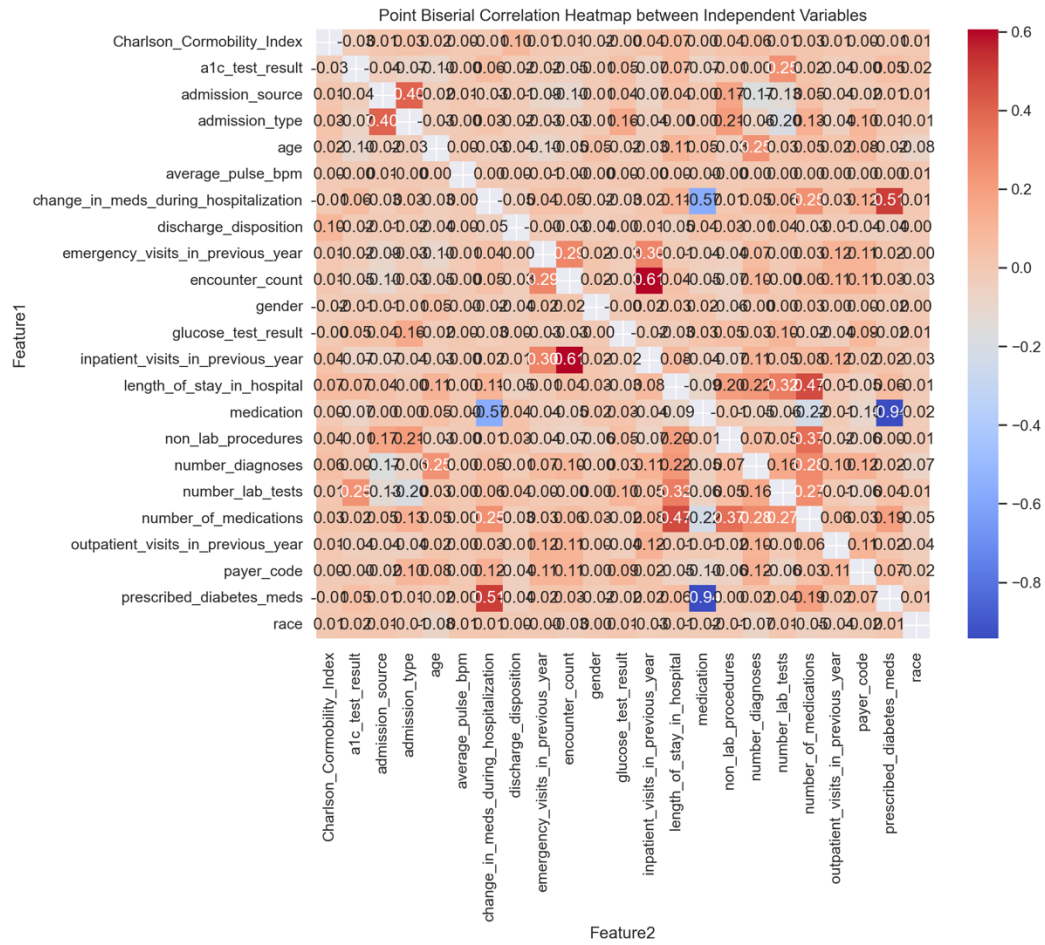


Figure 8 - Point Biserial Correlation – Multiclass

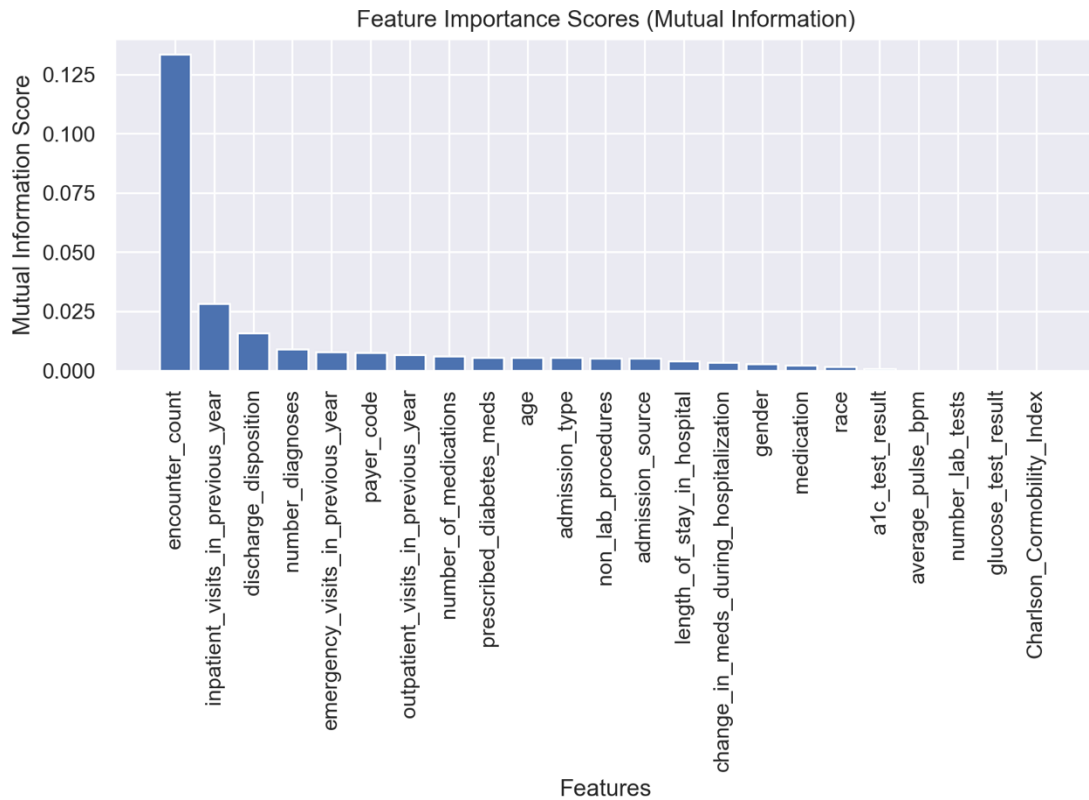


Figure 9 - Mutual Info Classification – Multiclass

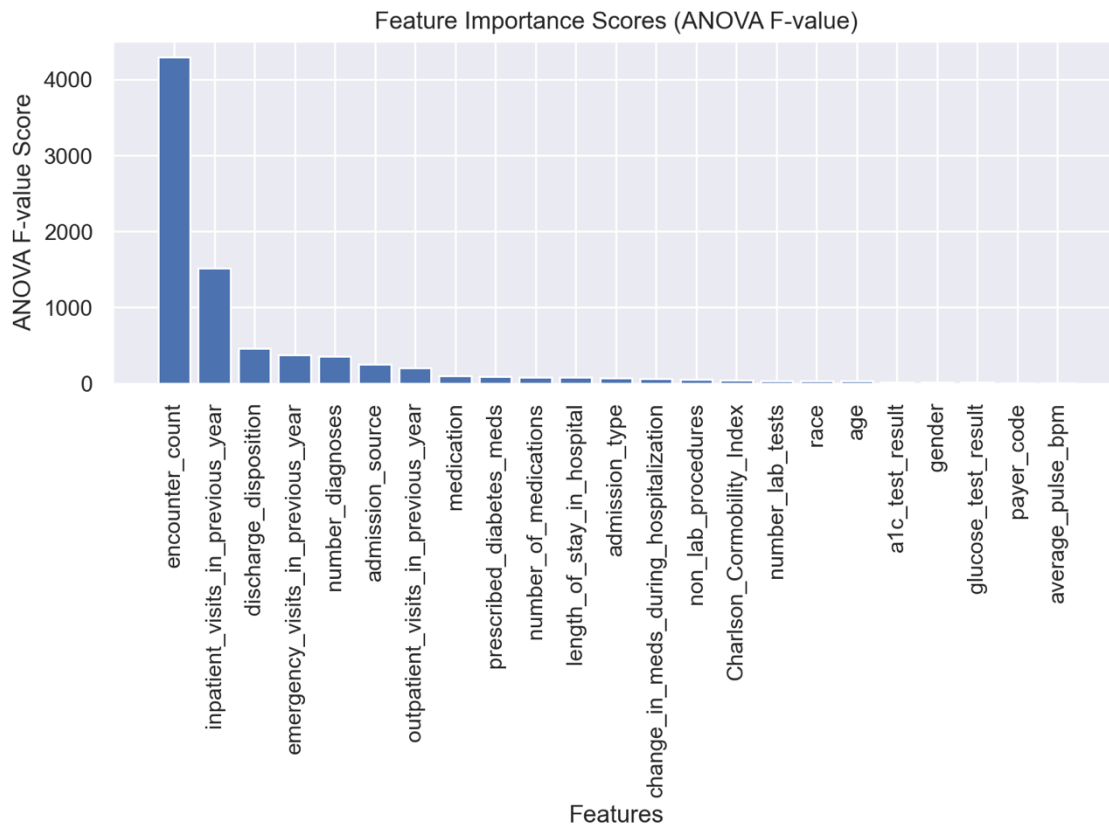


Figure 10 - ANOVA – Multiclass

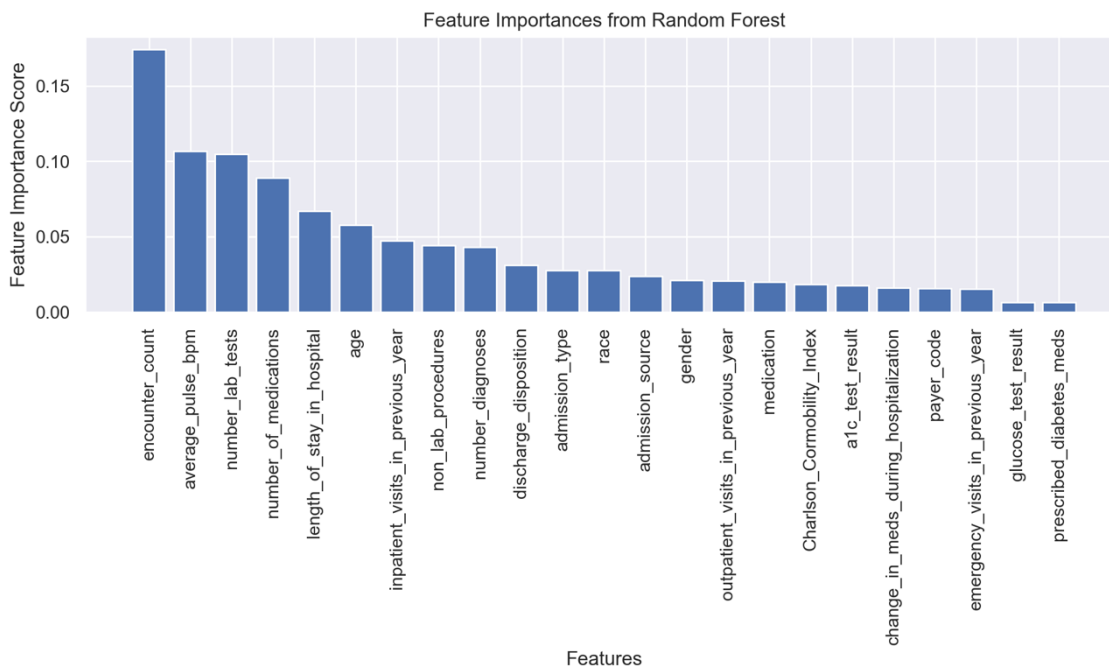


Figure 11 - Random Forests – Multiclass

	Variable	Lasso	Point Biserial	Kbest - MIC	Kbest - ANOVA	Random Forests	N° True	%True	Decision
0	race	True	True	True	False	True	4	0.8	Keep
1	gender	False	True	True	False	True	3	0.6	Drop
2	age	True	True	False	False	True	3	0.6	Drop
3	payer_code	True	True	True	False	True	4	0.8	Keep
4	outpatient_visits_in_previous_year	True	True	True	True	True	5	1.0	Keep
5	emergency_visits_in_previous_year	True	True	True	True	True	5	1.0	Keep
6	inpatient_visits_in_previous_year	True	True	True	True	True	5	1.0	Keep
7	admission_type	True	True	False	True	True	4	0.8	Keep
8	average_pulse_bpm	True	True	True	False	True	4	0.8	Keep
9	discharge_disposition	True	True	True	True	True	5	1.0	Keep
10	admission_source	True	True	True	True	True	5	1.0	Keep
11	length_of_stay_in_hospital	True	True	True	True	True	5	1.0	Keep
12	number_lab_tests	True	True	True	False	True	4	0.8	Keep
13	non_lab_procedures	True	True	False	False	True	3	0.6	Drop
14	number_of_medications	True	True	True	True	True	5	1.0	Keep
15	number_diagnoses	True	True	True	True	True	5	1.0	Keep
16	glucose_test_result	False	True	False	False	False	1	0.2	Drop
17	a1c_test_result	False	True	False	False	True	2	0.4	Drop
18	change_in_meds_during_hospitalization	True	True	False	True	True	4	0.8	Keep
19	prescribed_diabetes_meds	True	True	False	True	False	3	0.6	Drop
20	medication	True	False	True	True	True	4	0.8	Keep
21	Charlson_Cornobility_Index	True	True	True	False	True	4	0.8	Keep
22	encounter_count	True	True	True	True	True	5	1.0	Keep

Table 3 - Feature Selection Summary - Multiclass

	Time	Train	Validation	Precision	Recall
Logistic Regression	0.393	0.605	0.604	0.627	0.653
Gaussian Naive	0.011	0.581	0.578	0.582	0.614
KNN	0.004	0.721	0.599	0.59	0.614
Neural Network	21.506	0.662	0.644	0.648	0.682
Decision Tree	0.304	1.0	0.556	0.558	0.553
Random Forest	7.589	1.0	0.641	0.642	0.677
Adaboost	0.982	0.637	0.636	0.64	0.671
Gradient Boost	10.679	0.651	0.643	0.661	0.685

Table 4 - Multiclass Overview before Grid Search

	Time	Train	Validation	Precision	Recall
Logistic Regression	0.431	0.605	0.604	0.626	0.652
Gaussian Naive	0.012	0.581	0.578	0.582	0.614
KNN	0.005	0.721	0.599	0.59	0.614
Neural Network	17.355	0.668	0.65	0.652	0.682
Decision Tree	0.091	0.649	0.637	0.64	0.68
Random Forest	2.219	0.736	0.641	0.664	0.683
Adaboost	4.285	0.639	0.635	0.636	0.67
Gradient Boost	15.279	0.654	0.646	0.661	0.686
Stacking	178.431	0.658	0.646	0.66	0.685

Table 5 - Multiclass Overview

TABLE 1. ICD-9-CM and ICD-10 Coding Algorithms for Charlson Comorbidities			
Comorbidities	Deyo's ICD-9-CM +	ICD-10 *	Enhanced ICD-9-CM *
Myocardial infarction	410.x, 412.x	I21.x, I22.x, I25.2	410.x, 412.x
Congestive heart failure	428.x	I09.9, I11.0, I13.0, I13.2, I25.5, I42.0, I42.5-I42.9, I43.x, I50.x, P29.0	398.91, 402.01, 402.11, 402.91, 404.01, 404.03, 404.11, 404.13, 404.91, 404.93, 425.4-425.9, 428.x
Peripheral vascular disease	443.9, 441.x, 785.4, V43.4 Procedure 38.48	I70.x, I71.x, I73.1, I73.8, I73.9, I77.1, I79.0, I79.2, K55.1, K55.8, K55.9, Z95.8, Z95.9	093.0, 437.3, 440.x, 441.x, 443.1-443.9, 447.1, 557.1, 557.9, V43.4
Cerebrovascular disease	430.x-438.x	G45.x, G46.x, H34.0, I60.x-I69.x	362.34, 430.x-438.x
Dementia	290.x	F00.x-F03.x, F05.1, G30.x, G31.1	290.x, 294.1, 331.2
Chronic pulmonary disease	490.x-505.x, 506.4	I27.8, I27.9, J40.x-J47.x, J60.x-J67.x, J68.4, J70.1, J70.3	416.8, 416.9, 490.x-505.x, 506.4, 508.1, 508.8
Rheumatic disease	710.0, 710.1, 710.4, 714.0-714.2, 714.81, 725.x	M05.x, M06.x, M31.5, M32.x-M34.x, M35.1, M35.3, M36.0	446.5, 710.0-710.4, 714.0-714.2, 714.8, 725.x
Peptic ulcer disease	531.x-534.x	K25.x-K28.x	531.x-534.x
Mild liver disease	571.2, 571.4-571.6	B18.x, K70.0-K70.3, K70.9, K71.3-K71.5, K71.7, K73.x, K74.x, K76.0, K76.2-K76.4, K76.8, K76.9, Z94.4	070.22, 070.23, 070.32, 070.33, 070.44, 070.54, 070.6, 070.9, 570.x, 571.x, 573.3, 573.4, 573.8, 573.9, V42.7
Diabetes without chronic complication	250.0-250.3, 250.7	E10.0, E10.1, E10.6, E10.8, E10.9, E11.0, E11.1, E11.6, E11.8, E11.9, E12.0, E12.1, E12.6, E12.8, E12.9, E13.0, E13.1, E13.6, E13.8, E13.9, E14.0, E14.1, E14.6, E14.8, E14.9	250.0-250.3, 250.8, 250.9
Diabetes with chronic complication	250.4-250.6	E10.2-E10.5, E10.7, E11.2-E11.5, E11.7, E12.2-E12.5, E12.7, E13.2-E13.5, E13.7, E14.2-E14.5, E14.7	250.4-250.7
Hemiplegia or paraplegia	344.1, 342.x	G04.1, G11.4, G80.1, G80.2, G81.x, G82.x, G83.0-G83.4, G83.9	334.1, 342.x, 343.x, 344.0-344.6, 344.9
Renal disease	582.x, 583-583.7, 585.x, 586.x, 588.x	I12.0, I13.1, N03.2-N03.7, N05.2-N05.7, N18.x, N19.x, N25.0, Z49.0-Z49.2, Z94.0, Z99.2	403.01, 403.11, 403.91, 404.02, 404.03, 404.12, 404.13, 404.92, 404.93, 582.x, 583.0-583.7, 585.x, 586.x, 588.0, V42.0, V45.1, V56.x
Any malignancy, including lymphoma and leukemia, except malignant neoplasm of skin	140.x-172.x, 174.x-195.8, 200.x-208.x	C00.x-C26.x, C30.x-C34.x, C37.x-C41.x, C43.x, C45.x-C58.x, C60.x-C76.x, C81.x-C85.x, C88.x, C90.x-C97.x	140.x-172.x, 174.x-195.8, 200.x-208.x, 238.6
Moderate or severe liver disease	456.0-456.21, 572.2-572.8	I85.0, I85.9, I86.4, I98.2, K70.4, K71.1, K72.1, K72.9, K76.5, K76.6, K76.7	456.0-456.2, 572.2-572.8
Metastatic solid tumor	196.x-199.1	C77.x-C80.x	196.x-199.x
AIDS/HIV	042.x-044.x	B20.x-B22.x, B24.x	042.x-044.x

Table 6 - Coding algorithms for Charlson Comorbidities, Adapted from "Coding Algorithms for ICD-9-CM and ICD-10" by Quan H, Sundararajan V, Halfon P, et al, 2005